

Final Assignment

Data in Python course, Spring 25/26

Ver. 1.00

Your task in this assignment is to create a program for data analysis. The program must be written in Python and delivered as a Python package that can be installed via pip. You also have to attach to that package the results of profiling your code's execution (you do not have to optimise the code; just submitting the profiling results and pointing out possible bottlenecks is enough). You should use at least one of the libraries shown in the lecture (*numpy* or *pandas*) for data analysis.

Your Program should be implemented as a library that provides all essential functions. A simple Python program and a Jupyter notebook should also be used with that library. The script should use *argparse* to read the parameters (such as the output file name or path to a directory with data files), and the analysis results should be written to a file (specified as a program argument). Your library must not contain any hardcoded paths to any data files. Unit tests should also be provided with your code.

Assignment versions

There are two possible versions of the data analysis task:

- the one described below (the canonical one),
- your own (has to be compatible with the canonical requirements and has to be accepted ahead of time by the teacher in your group).

The canonical assignment

Your task is to analyse the language used in writing by a Master of Literature (like Mickiewicz, Shakespeare, or Cervantes). Your program should be prepared for one language (Polish, English, Spanish, ...), but it should be easy to adapt it to any other similar language.

Your program has to:

- Read in a dictionary of a language (the `---dictionary` parameter) - a text file containing all forms of words in a language. Each line should contain the word, followed by all forms separated by ", ". A dictionary for the Polish language is added to the assignment.
- According to the `--dictionary-stats` command line option (if it is present), the program should display statistics of the dictionary, like (at least):
 - number of lines,
 - number of words (i.e. all words in the file),
 - number of unique words,
 - top 10 words with the most occurrences in the file (if there is a smaller number of duplicates, then only those found, if there are more words with the same occurrence count as the ones in the first 10th, display all of them).
 - letters (of a proper language) with counters of occurrences in the words, then the other characters found in words in the dictionary (also with counters).
- Read all given text files with the works of the Master (the `--works` command parameter). The file names should be comma-separated on the command line, and there should be at least one of them.
- According to the `--dictionary-stats` command line option (if it is present), the program should display statistics of the files, like (at least):
 - number of files,
 - number of lines,
 - number of words (i.e. all words in the file),

- number of unique words,
- top 10 words with the most occurrences in the file (if there is a smaller number of duplicates, then only those found, if there are more words with the same occurrence count as the ones in the first 10th, display all of them).
- letters (of a proper language) with counters of occurrences in the words, then the other characters found in files (also with counters).
- If more than one file was given, the statistics should be given as a total for all files, and also for each separate file (for one file, total is the same as for each - the only one - file, so they should not be displayed twice).
- When extracting words from files, it would be useful to strip various punctuation characters, such as commas. By experimenting, determine which characters should be eliminated from the works of your selected Master.
- According to the `--no-words` command line parameter, analyse which words of the Master files do not exist in the current dictionary of the Master's language. Display them all along with their occurrences.
- According to the `--frequencies` (if present, it has an option value `n`) command-line parameter, calculate for each of Master's tasks the `n` most frequent words (and display them with counters). Based on these data (and maybe other) propose a function (criterion) returning as a result a number from the range 0..100, approximating the similarity of style of those works (how likely both files were written by the same author). Illustrate the use of that function with graphs (comparing files no. 1 to other files, then comparing file no. 2 to the rest, etc.).
- Prepare a Jupyter notebook, which summarises your statistics and charts. describe there your comparison criterion and describe your conclusions about the language (and words) used in the Master's works and the current language.

All displayed lists should be sorted by the number of occurrences, then alphabetically.

Evaluation rules:

- Explanation of design decisions during the exam.
- Proper division of code into packages and modules.
- Code quality - proper names for functions and variables. Short functions. We suggest using automated tools like `pylint` or `flake` to check the code. We also suggest reading <https://www.python.org/dev/peps/pep-0008/>
- The possibility to install code as a package using ``pip install path_to_package_directory``
- The code should be provided in a repository. Putting obsolete files in the repository can lower the grade.
- The possibility to install should be introduced as a Pull Request (Merge Request) to the master (or main) branch.
- Code coverage with tests.
- The usage of libraries, which were shown during class.
- Code profiler output, and its explanation.

Important note

During the exam, you will be asked to make a minor amendment to your code. It will not be complicated, but failing to make this amendment will result in failing the exam (regardless of the rest of the code provided). We have to be sure that you are the authors of the presented solution.

If you look for inspiration for your own task, you could check this source of data:

- <https://api.um.warszawa.pl/#>
- <https://stat.gov.pl/>
- <https://dane.gov.pl/pl>

